

醫療病歷輔助查詢系統 — 技術說明文件

本文件說明系統的完整運作流程、相似度計算方式與信心等級判斷邏輯，適合對外介紹使用。

一、系統定位

本系統是一套全離線的醫療 RAG（Retrieval-Augmented Generation）系統。

RAG 的核心概念是：先搜尋、再生成。不讓 AI

憑空回答，而是先從資料庫找出相關依據，再根據這些依據產生回答，大幅降低 AI 捏造資訊（幻覺）的風險。

二、系統完整流程

一個問題從輸入到輸出，共經過以下 8 個階段：



① PHI 偵測（輸入端）

做什麼：掃描使用者輸入，偵測是否包含個人識別資訊（PHI）。

系統使用正規表示式偵測：

PHI 類型	偵測規則範例
台灣身分證	一個英文字母 + 數字 1 或 2 + 8 位數字
手機號碼	09XX-XXX-XXX 格式
台灣地址	含「市」「區」「路」「街」「號」的字串
出生日期	民國或西元日期格式

若偵測到 PHI，系統會：

- 將個資替換為佔位符（例如 [ID_NUMBER]、[PHONE]）再繼續處理
- 在回答旁顯示警示訊息

② Embedding（語意向量化）

做什麼：將使用者輸入的文字轉換成一個數字向量，捕捉語意。

使用模型：paraphrase-multilingual-MiniLM-L12-v2（HuggingFace，支援 50+ 語言，含繁體中文與醫學英文混合）

技術細節：

- 輸出維度：384 維，也就是每段文字被壓縮成一個有 384 個數字的向量
- 語意相近的文字，在向量空間中的距離也相近
- 向量做 L2 歸一化（normalize），長度統一為 1，方便後續計算

直觀比喻：把「病患胸痛、呼吸困難、心電圖 ST

上升」這段描述，壓縮成空間中的一個座標點。病情越相似的描述，座標點越靠近。

③ FAISS 向量搜尋（第一層排序）

做什麼：在所有已索引的病例向量中，找出距離最近的 top-20 候選。

索引建立（事前）：

- 每筆病歷的「主訴 + 現病史 + 診斷」合併為一個 query chunk
- 將所有 query chunk 都 embed 成向量，存入 FAISS 索引
- 這樣的切割設計讓搜尋時的語意最集中（症狀找症狀）

查詢時：

- 使用 IndexFlatIP（向量內積，Inner Product）
- 因為向量已歸一化，內積 = 餘弦相似度（cosine similarity）
- 計算使用者向量和每筆病例向量的餘弦相似度，取前 20 名

相似度分數說明：

介面上顯示的百分比（例如「63.2%」）就是這個餘弦相似度分數，轉換為百分比表示。

餘弦相似度的意義：

分數	意義
100%	完全相同的語意
75%+	病情描述高度相似
60~75%	有相關性，但存在差異
60% 以下	相關性低，結果可靠性下降

④ Cross-encoder Reranker（第二層排序）

做什麼：對 FAISS 找出的 top-20 候選，用更精準的模型重新排序，取 top-5。

為什麼需要兩層？

FAISS 用的

bi-encoder（雙編碼器）把查詢和文件分別壓縮成向量再比較，速度快但精準度有限——有時「用詞相似但病情不同」的案例會排很前面，「用詞不同但病情真正相似」的卻被排後面。

使用模型：cross-encoder/mmarco-mMiniLMv2-L12-H384-v1（多語言 cross-encoder）

運作方式：

- 把每個（使用者查詢, 候選病例）配對，一起丟進 cross-encoder
- cross-encoder 同時看到兩段文字，能做更細緻的語意比對
- 對 20 個配對各打一個相關性分數，重新排序，保留前 5 名

兩層設計的好處：

- FAISS 快速從幾萬筆中找出 20 個候選（毫秒級）
- Reranker 精準地在 20 個中選出最佳 5 個（秒級）
- 資料量增加時只需調高 FAISS 的召回數（例如改為 top-50），不需修改架構

⑤ 相似度閾值檢查

做什麼：判斷找到的病例是否有參考價值。

若 top-1 的 FAISS 相似度 < 0.5 （50%），視為資料庫中無充足相似病例，直接回傳提示訊息，不啟動 LLM 生成。

這是避免 AI 在沒有依據的情況下憑空生成的第一道防線。

⑥ LLM 生成回答

做什麼：根據 top-5 相似病例的詳細資料，生成自然語言回答。

Small-to-Big Retrieval 設計：

- 前面用「主訴 + 現病史 + 診斷」做搜尋（小 chunk，語意集中）
- 生成時改用「診斷 + 治療計畫 + 用藥 + 結果 + 追蹤」（大 chunk，資訊完整）
- 搜尋精準，生成詳細

使用模型：meta-llama/Llama-3.2-3B-Instruct（全離線，4-bit 量化，VRAM $\approx$ 2.5GB）

System Prompt 設計（強制規則）：

- 只能根據提供的病例資料回答，不得憑空捏造
- 不得輸出任何病患個人識別資訊
- 必須標注「根據相似病例顯示...」而非直接斷言
- 若資料不足，明確說明「資料庫中無充足相似病例」
- 涉及用藥劑量必須強調需經醫師確認

⑦ 信心等級評估

做什麼：從兩個維度評估這次回答的可信度，輸出 HIGH / MEDIUM / LOW 三個等級。

兩個評估維度

維度一：Retrieval Score（檢索相似度）

- 即步驟③的 FAISS cosine similarity
- 反映「找到的病例和當前問題有多相似」

維度二：Faithfulness Score（忠實度）

- 計算公式：回答中出現在病例 context 的醫療詞彙數 ÷ 回答中所有醫療詞彙數
- 反映「LLM 的回答有多少內容是有資料依據的」
- 醫療詞彙包含：疾病名、症狀名、藥物名、檢查項目、處置方式等

信心等級判斷邏輯

```
if retrieval_score ≥ 0.75
AND faithfulness ≥ 0.60
AND 回答不含具體劑量數字
AND 回答不含強烈建議詞
→ HIGH（綠色）
```

```
elif retrieval_score ≥ 0.60
OR faithfulness ≥ 0.50
AND 回答不含具體劑量數字
AND 回答不含強烈建議詞
→ MEDIUM（黃色）
```

```
else
→ LOW（橙色）
```

自動觸發 LOW 的情況

觸發條件	原因
含具體劑量（如 500mg、5mg/kg）	劑量錯誤風險高，必須人工確認
含強烈建議詞（「建議立即」「必須」「緊急」）	過度確定性的表述，需醫師判斷
Retrieval Score < 0.6	找到的病例參考性不足

⑧ PHI 掃描（輸出端）

做什麼：在回答送出前，再掃描一次 LLM 的輸出，確保沒有個資洩漏。

這是最後一道防線，防止 LLM 在生成過程中從 context 重組出 PHI。

四、去識別化：三層防護架構

資料在整個生命週期的不同階段，都有不同的防護措施：



五、為什麼相似度不是 100%？

即使查詢和某個歷史病例幾乎一樣，相似度也很少超過 85%，原因有三：

- 用詞差異：不同醫師對同一病情的描述方式不同（「胸骨後壓迫感」vs「胸悶」）
- 資訊量差異：使用者輸入的症狀描述通常比完整病歷精簡
- 向量壓縮損失：384 維向量無法完整保留所有語意細節

這是正常現象，60% 以上即有參考價值，75% 以上為高度相關。

六、技術選型一覽

元件	選擇	說明
向量資料庫	FAISS	純本地，無需伺服器，Facebook AI 開源
Embedding 模型	paraphrase-multilingual-MiniLM-L12-v2	支援 50+ 語言，384 維，輕量
Reranker	mmarco-mMiniLMv2-L12-H384-v1	多語言 cross-encoder，精確重排序

LLM

Web UI	FastAPI + HTML/Jinja2	Meta 開源，4-bit 量化後約 2.5GB VRAM
運行環境	全離線	自訂版面，支援查詢紀錄，後端 API 與前端模板分離 無資料外傳，符合醫療隱私需求

七、系統限制與注意事項

- 資料庫規模：目前索引 50 筆合成病歷，實際部署應擴充為真實去識別化病歷
- LLM 規模：3B 模型的醫療知識深度有限，正式場景建議使用醫療專用大模型
- 僅供輔助：系統提供的資訊僅供醫療人員參考，所有臨床決策需由執業醫師判斷
- 合成資料：本練習版所有病歷均為人工合成，與任何真實病患無關